Report for the Course on Cyber-Physical Systems and IoT Security

Second mid-term project

Professional Security Corporation

Members

Emanuele Artusi | 2198545

Filippo Bellon | 2199368

Reference paper

An Anomaly-Based IDS for Detecting Attacks in RPL-Based Internet of Things

A.Y. 2025/26

Contents

1 Introduction

The Internet of Things (IoT) has evolved into a widespread paradigm connecting large numbers of resource-constrained devices and enabling the development of intelligent cyber-physical systems. Many IoT deployments rely on Low-Power and Lossy Networks (LLNs), where nodes operate under strict limitations in terms of processing power, memory, energy, and communication bandwidth. Technologies such as IPv6 over Low-Power Wireless Personal Area Networks (6LoWPAN) enable Internet Protocol Version 6 (IPv6) communication in these constrained environments, while the Routing Protocol for Low-Power and Lossy Networks (RPL), standardized by the Internet Engineering Task Force (IETF), provides distributed routing and topology formation.

The characteristics of LLNs and the distributed nature of RPL introduce significant security challenges. In particular, compromised internal nodes can legitimately participate in the routing process while manipulating RPL control information, making conventional security mechanisms insufficient against internal routing attacks [1]. Among these attacks, Sinkhole and Blackhole attacks are particularly relevant. A Sinkhole node can advertise an artificially attractive routing metric to attract network traffic through itself, while a Blackhole node can exploit its position in the routing topology to selectively or completely discard forwarded packets.

This work investigates the behavior of a RPL network under Sinkhole and Blackhole attacks and evaluates their impact on network performance. In addition, a lightweight log-based Intrusion Detection System (IDS) is developed to identify routing anomalies by monitoring RPL control messages and detecting inconsistencies in node and parent ranks.

2 Background

This section introduces the concepts required to understand the experimental work.

2.1 The RPL routing protocol

The RPL is a distance-vector routing protocol designed for resource-constrained LLNs. RPL organizes nodes into a Destination-Oriented Directed Acyclic Graph (DODAG), whose topology is rooted at a node providing connectivity towards an external network.

2.1.1 DODAG construction and topology management

Each node in a DODAG is assigned a *Rank*, which represents its position relative to the root. The root has the lowest Rank, while nodes farther from the root generally have higher Rank values. Rank is used to maintain the directed acyclic structure of the DODAG and to support loop avoidance.

The selection of a preferred parent and the computation of the resulting Rank are determined by an *Objective Function (OF)*, which evaluates the routing cost associated with neighboring nodes. Two OFs commonly associated with RPL are:

- **Objective Function Zero (OF0):** primarily considers the number of hops towards the root when selecting a parent;
- **Minimum Rank with Hysteresis Objective Function (MRHOF):** considers link-quality metrics such as the Expected Transmission Count (ETX) to compute the path cost and applies hysteresis to limit unnecessary parent changes.

In the RPL implementation considered in this work, MRHOF is used for parent selection. The advertised Rank of a neighbor therefore represents an important component of the routing cost considered when evaluating potential parents.

2.1.2 RPL control messages

RPL uses Internet Control Message Protocol for IPv6 (ICMPv6) control messages to construct and maintain the DODAG:

- **DODAG Information Object (DIO):** advertises DODAG information, including the current Rank, version, and other configuration parameters. Nodes use received DIOs to discover potential parents;
- **DODAG Information Solicitation (DIS):** requests DIO messages from neighboring nodes, for example when joining or recovering from a disconnected state;

- **Destination Advertisement Object (DAO):** advertises routing information towards the root, allowing downward routes to be maintained;
- **Destination Advertisement Object Acknowledgment (DAO-ACK):** optionally acknowledges the reception of a DAO.

Among these messages, the DIO is particularly relevant to the attacks considered in this work, since it contains the Rank advertised by a node and is therefore involved in parent selection.

2.2 Routing attacks in RPL networks

The distributed nature of RPL allows a compromised internal node to participate in the routing protocol while providing malicious routing information. Two routing attacks are considered in this project: Blackhole and Sinkhole.

2.2.1 Blackhole attack

A Blackhole attack consists of a compromised routing node that discards packets that it is expected to forward instead of delivering them towards their destination. The malicious node may continue to participate normally in RPL topology formation, while disrupting data traffic that is routed through it.

The attack primarily affects packet forwarding and can therefore result in packet loss and reduced network delivery performance.

2.2.2 Sinkhole attack

A Sinkhole attack attempts to attract network traffic by making a compromised node appear to provide a highly favorable route towards the DODAG root. In RPL, this can be achieved by manipulating the Rank advertised in DIO messages.

The attacker advertises a Rank lower than its actual Rank, causing neighboring nodes to potentially consider it a more attractive parent. Once selected as a preferred parent, the malicious node can attract traffic originating from other nodes in its neighborhood.

The effectiveness of the attack depends on the routing OF, link metrics, and parent-selection mechanisms. In particular, the hysteresis mechanism of MRHOF can delay parent changes when the new route does not provide a sufficiently large improvement.

A Sinkhole attack can also be combined with a Blackhole attack, where the Sinkhole redirects traffic towards the compromised node and the Blackhole subsequently discards it.

2.3 IDS in IoT

IDS provide an additional layer of defense by monitoring network activity and identifying behavior that deviates from expected protocol operation.

IDS approaches can be deployed directly on network nodes or at a centralized monitoring point. They can also rely on different detection strategies, such as predefined protocol rules or the identification of anomalous behavior.

In this work, the IDS is based on RPL routing information and focuses on identifying inconsistencies in the observed routing hierarchy. In particular, Rank information advertised through DIO messages and parent relationships reported through DAO messages can be used to assess whether the resulting topology is consistent with the expected RPL hierarchy.

3 Objectives

The main objective of this project is to investigate the impact of routing attacks on RPL-based IoT networks and to evaluate the feasibility of detecting such attacks through routing information.

The specific objectives are:

- **Analyze RPL behavior:** study the construction and evolution of the DODAG under normal operating conditions;
- **Evaluate routing attacks:** implement and analyze Blackhole and Sinkhole attacks and observe their effects on the routing topology and packet delivery;

- **Compare network performance:** evaluate the impact of the attacks using metrics such as Packet Delivery Ratio (PDR), End-to-End Delay, and Throughput;
- **Develop a lightweight IDS:** analyze Contiki-NG execution logs to identify routing anomalies, with particular focus on inconsistencies between node and parent Rank values;
- **Assess detection capability:** evaluate whether the proposed IDS can identify the routing inconsistencies introduced by a Sinkhole attack.

4 System setup

4.1 Experimental environment

We deployed the experimental environment on a Windows 10 host using Windows Subsystem for Linux 2 (WSL2). We used an Ubuntu Linux environment to build and execute the Contiki-NG source code and its simulation tools.

We simulated the RPL network using Cooja, the network simulator included with Contiki-NG. Cooja allowed us to execute multiple Contiki-NG motes simultaneously while providing access to their network activity and execution logs.

To run the graphical Cooja interface through WSL2, we configured the display variable before launching the simulator:

```
export DISPLAY=:0
```

We then started the simulator from the Contiki-NG directory using:

```
cd tools/cooja
./gradlew run
```

We used the `RPL-UDP` example as the basis for our experiments. We created custom Cooja simulation configurations to define the network topology, participating motes, and attack scenarios. The simulations produced mote execution logs, which we subsequently analyzed to evaluate network performance and detect routing anomalies.

4.2 Implementation modifications

To implement the two attacks considered in our experiments, we modified selected components of the Contiki-NG networking and RPL stack. The modifications were kept localized to the routing and forwarding mechanisms, while the normal application and RPL behavior was preserved whenever possible. You can find the complete source code of the modified Contiki-NG files in the `Source/Contiki patches` directory of the accompanying repository.

4.2.1 Blackhole attack

We implemented the Blackhole attack at the IPv6 forwarding layer in `os/net/ipv6/uip6.c`. The attack is enabled through a compile-time configuration and is associated with a specific simulated mote (as for the Sinkhole attack):

```
#define BLACKHOLE_ID 5
#define B_ATTACK_ACTIVE 1
```

When the malicious mote forwards a packet, we inspect the actual transport protocol. This also accounts for IPv6 packets containing a Hop-by-Hop extension header:

```
if(B_ATTACK_ACTIVE && simMoteID == BLACKHOLE_ID) {
  uint8_t real_proto = UIP_IP_BUF->proto;
  if(real_proto == UIP_PROTO_HBHO) {
    struct uip_ext_hdr *ext_hdr;
    ext_hdr = (struct uip_ext_hdr *)
      ((uint8_t *)UIP_IP_BUF + UIP_IPH_LEN);
    real_proto = ext_hdr->next;
```

```
  }
  if(real_proto == UIP_PROTO_UDP) {
    LOG_INFO("BLACKHOLE DROP UDP forwarding\n");
    goto drop;
  }
}
```

The check was inserted in the IPv6 forwarding path, before the packet is sent to the next hop. Consequently, User Datagram Protocol (UDP) packets that reach the malicious node for forwarding are discarded, while the node can continue to participate in RPL and generate its own traffic.

We also added a dedicated `blackhole.c` application for the malicious mote. Its main purpose is to initialize the node and its participation in the routing stack:

```
PROCESS(blackhole_process, "Blackhole node");
AUTOSTART_PROCESSES(&blackhole_process);
PROCESS_THREAD(blackhole_process, ev, data)
{
  PROCESS_BEGIN();
  LOG_INFO("Blackhole started\n");
  NETSTACK_ROUTING.init();
  while(1) {
    PROCESS_WAIT_EVENT();
  }
  PROCESS_END();
}
```

The attack-specific forwarding behavior therefore remains in the IPv6 stack, rather than being implemented as an application-level packet drop.

In addition to the standard Blackhole behavior described above, we also implemented an *Aggressive Blackhole* variant in `os/net/ipv6/tcpip.c`. This variant modifies the forwarding behavior more aggressively in order to discard every packet reaching the compromised node. The implementation was retained as an additional attack configuration, but it was not evaluated in the experimental scenarios reported in this work because it is less realistic and more easily detectable than the standard Blackhole attack, since it will break the DODAG construction and prevent the malicious node from participating in the routing process.

4.2.2 Sinkhole attack

We implemented the Sinkhole attack by modifying the RPL DIO generation mechanism in `os/net/routing/rpl-lite/rpl-icmp6.c`. Normally, the advertised Rank corresponds to the node's actual Rank:

```
uint16_t advertised_rank = curr_instance.dag.rank;
```

For the malicious node, the attack is activated by replacing the advertised Rank with `ROOT_RANK`:

```
if(simMoteID == SINKHOLE_ID &&
   curr_instance.dag.rank != ROOT_RANK /*&&
   curr_instance.dag.rank != RPL_INFINITE_RANK*/) {
  advertised_rank = ROOT_RANK /*+
                    curr_instance.min_hoprankinc*/;
  LOG_INFO("SINKHOLE ACTIVE fake_rank=%u "
           "real_rank=%u\n",
           advertised_rank,
           curr_instance.dag.rank);
}
```

The commented conditions represent the behavior that would be preferable for a stealthier attack. In particular, excluding `RPL_INFINITE_RANK` would prevent the attack from becoming active while the DODAG is still being constructed, when the node has not yet selected a valid parent. Similarly, adding `curr_instance.min_hoprankinc` to the advertised Rank would avoid advertising exactly the root Rank while still providing an artificially attractive routing metric.

For the experiments, however, these conditions and the Rank increment were left commented out. This makes the Sinkhole behavior more pronounced and therefore easier to observe and analyze in the simulation logs.

The malicious node therefore advertises a Rank corresponding to the root while maintaining its actual Rank internally. This directly affects the routing information received by neighboring nodes through DIO messages.

We also modified the Metric Container of the DIO. When MRHOF uses ETX, the Sinkhole advertises a fixed ETX value corresponding to 1.0 instead of its actual path metric:

```
uint16_t fake_etx = curr_instance.mc.obj.etx;
if(S_ATTACK_ACTIVE &&
   simMoteID == SINKHOLE_ID) {
  fake_etx = 128;
  LOG_INFO("SINKHOLE: advertising fake ETX %u "
           "(real %u)\n",
           fake_etx,
           curr_instance.mc.obj.etx);
}
```

The two manipulated values, Rank and ETX, are therefore inserted directly into the RPL control information advertised by the malicious node.

4.2.3 MRHOF path-cost modification

Since the experiments use the MRHOF, we also modified its path-cost calculation in `os/net/routing/rpl-lite/rpl-mrhof.c`.

We introduced a configurable multiplier for the link metric:

```
#define LINK_METRIC_MULTIPLIER 0
```

The original path-cost calculation was modified as follows:

```
return MIN((uint32_t)base +
           link_metric_to_rank(nbr_link_metric(nbr))
           * LINK_METRIC_MULTIPLIER,
           0xffff);
```

With the multiplier set to zero, the link-metric contribution is removed from the resulting path cost:

$$C_{\text{path}} = R_{\text{parent}} \tag{1}$$

This configuration allowed us to isolate the effect of the Rank advertised by the Sinkhole and reduce the influence of link-quality variations during the experiments.

4.2.4 Build and logging configuration

We modified the `examples/rpl-udp/Makefile` to include the dedicated Blackhole node and to enable the logging required during the experiments:

```
CONTIKI_PROJECT = udp-client udp-server blackhole
all: $(CONTIKI_PROJECT)
CONTIKI=../..
CFLAGS += -DLOG_CONF_LEVEL_RPL=LOG_LEVEL_DBG
CFLAGS += -DBLACKHOLE_NODE
include $(CONTIKI)/Makefile.include
```

We also enabled debug-level logging in the modified source files. This provided the RPL and forwarding information required to reconstruct the routing behavior from the simulation logs.

4.3 IDS implementation

We developed two lightweight, centralized IDSs in Python, operating on the execution logs generated by the Contiki-NG simulations. The two IDSs target the different characteristics of the attacks considered in this work.

4.3.1 Blackhole IDS

The Blackhole IDS, implemented in `Source/IDS/IDS-blackhole.py`, monitors packet forwarding activity. It counts packets received for forwarding and packets subsequently forwarded by each node:

```
received_forward[node] += 1
forwarded[node] += 1
```

For each node, a forwarding ratio is calculated. A node is flagged when it receives more than five packets and forwards less than 60% of them. The IDS also computes PDR, end-to-end delay, and throughput and stores the results in `ids_blackhole_results.csv`.

4.3.2 Sinkhole IDS

The Sinkhole IDS, implemented in `Source/IDS/IDS-sinkhole.py`, reconstructs the RPL topology from DIO and DAO messages by extracting node Ranks and observed parent relationships.

The main detection rule checks the consistency between a node and its parent:

```
if parent_rank >= child_rank:
    alerts.append(...)
```

Such a relationship violates the expected RPL Rank hierarchy and is reported as a potential routing anomaly. The IDS also records significant Rank changes as secondary indicators, while accounting for the normal Rank initialization that occurs during DODAG formation. The observed Rank history is exported to `ids_sinkhole_results.csv`.

5 Experiments

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

6 Results

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Acronyms

6LoWPAN IPv6 over Low-Power Wireless Personal Area Networks. 2

DAO Destination Advertisement Object. 3, 7

DAO-ACK Destination Advertisement Object Acknowledgment. 3

DIO DODAG Information Object. 2, 3, 5–7

DIS DODAG Information Solicitation. 2

DODAG Destination-Oriented Directed Acyclic Graph. 2, 3, 5, 7

ETX Expected Transmission Count. 2, 6

ICMPv6 Internet Control Message Protocol for IPv6. 2

IDS Intrusion Detection System. 2–4, 6, 7

IETF Internet Engineering Task Force. 2

IoT Internet of Things. 2, 3

IPv6 Internet Protocol Version 6. 2, 4, 5

LLN Low-Power and Lossy Network. 2

MRHOF Minimum Rank with Hysteresis Objective Function. 2, 3, 6

OF Objective Function. 2, 3

OF0 Objective Function Zero. 2

PDR Packet Delivery Ratio. 4, 7

RPL Routing Protocol for Low-Power and Lossy Networks. 2–7

UDP User Datagram Protocol. 5

WSL2 Windows Subsystem for Linux 2. 4

References

[1] Behnam Farzaneh, Mohammad Ali Montazeri, and Shahram Jamali. An anomaly-based ids for detecting attacks in rpl-based internet of things. pages 61–66, 2019.